Student Name: Steven Peters, 23021262
Course: 159.251 - Software Design and Construction
Assignment: Assignment 2

Performance Analysis Report

1. Introduction

This report presents the performance analysis of custom Log4j components: MemAppender and VelocityLayout. The analysis compares their performance against built-in Log4j components under various configurations

Test Methodology

- Number of log messages: 10,000 per test
- MaxSize values tested: 1, 10, 100, 1,000, 10,000, 1,000,000
- Measurements: Time (milliseconds) and Memory (kilobytes)
- Profiling tool: VisualVM
- Test framework: JUnit 5 with parameterized tests

2. Appender Performance Comparison

2.1 MemAppender with ArrayList vs LinkedList

This section compares the performance of MemAppender using two different List implementations

Results Summary Table

MaxSize	ArrayList Time (ms)	ArrayList Memory (KB)	LinkedList Time (ms)	LinkedList Memory (KB)	Logs Discarded
1	8	1589	68	2629	9999
10	2	1364	6	1956	9990
100	3	1364	3	1956	9900
10000	2	2102	2	1956	0
1000000	2	2102	2	1956	0

Analysis

Key Findings:

- ArrayList performed better than LinkedList overall
- The performance gap was most significant at maxSize = 1 where ArrayList was 60ms faster compared to LinkedList
- As maxSize increased, the performance difference became far less significant

Explanation:
The performance difference is due to the `remove(0)` operation required when maxSize is exceeded:

- ArrayList: Removing from index 0 requires shifting all remaining elements, resulting in O(n) complexity
- LinkedList: Removing from the head only requires updating the head pointer, resulting in O(1) complexity

However, despite LinkedList's theoretical advantage for remove(0) operations, ArrayList performed better in practice. This is due to a few factors:

- Cache: ArrayList stores elements in contiguous memory
- Memory overhead: LinkedList requires additional Node objects with next/previous pointers for each element

2.2 MemAppender vs ConsoleAppender vs FileAppender

Results Summary Table

Appender Type	MaxSize	Time (ms)	Memory (KB)	Notes
MemAppender (ArrayList)	10000	2	2102	Stores in memory
ConsoleAppender	10000	20	9782	Writes to console
FileAppender	10000	11	4094	Writes to file

Analysis

Key Findings:

- MemAppender performed the fastest, followed by FileAppender with ConsoleAppender being the slowest

Explanation:

- MemAppender performance is the fastest because it only performs in-memory operations. Adding log events to a List and managing the maxSize requires no I/O operations making it extremely fast
- ConsoleAppender performance is slower than MemAppender because it requires I/O operations to write to the console via System.out
- FileAppender writes logs to disk, which involves file system operations, and are orders of magnitude slower than memory access

3. Layout Performance Comparison

3.1 VelocityLayout vs PatternLayout

Results Summary Table

Layout Type	MaxSize	Time (ms)	Memory (KB)	Logs Stored
VelocityLayout	1	185	5661	1
PatternLayout	1	21	2405	1
VelocityLayout	100	64	4188	100
PatternLayout	100	7	2180	100
VelocityLayout	10000	522	23679	10000
PatternLayout	10000	29	4789	10000

Analysis

Key Findings:

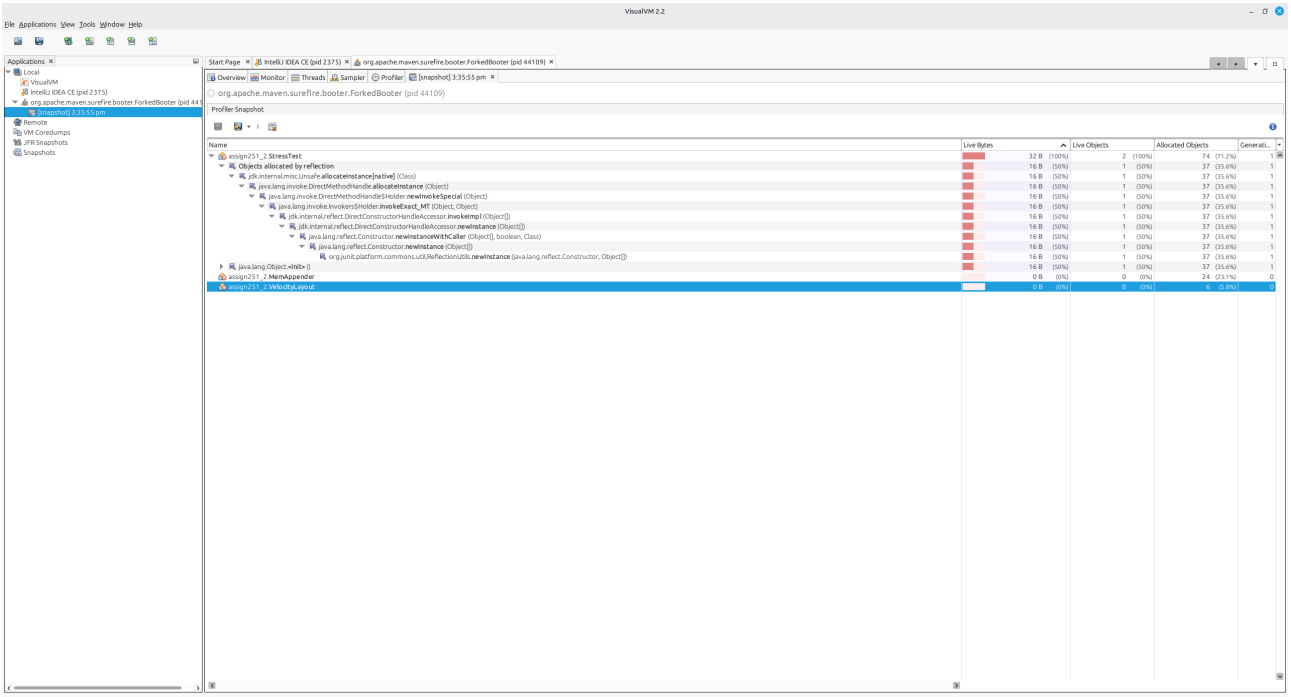
- PatternLayout was significantly faster than VelocityLayout
- The performance difference was across all maxSize values
- Memory usage between the two layouts was also higher with VelocityLayout

Explanation:
The performance difference is primarily due to:

- PatternLayout uses pre-compiled patterns developed and refined over many years in Log4j
- VelocityLayout creates a new VelocityContext for each log event and calls `Velocity.evaluate()`, which involves template parsing and evaluation overhead

While VelocityLayout offers more flexibility with Velocity's template syntax, this comes at a performance cost as shown above

Memory utlization:



CPU utilization: